



南开大学  
Nankai University

# Java 第一次实验报告

## Java 实现 3D 福彩模拟

学 院：软件学院

姓 名：黄子豪

学 号：2413989

2025~2026 第二学期

使用 L<sup>A</sup>T<sub>E</sub>X 撰写于 2026 年 4 月 7 日

# 目 录

|   |                         |    |
|---|-------------------------|----|
| 1 | 实验目的 . . . . .          | 2  |
| 2 | 实验内容与需求分析 . . . . .     | 2  |
| 3 | 总体设计 . . . . .          | 2  |
|   | 3.1 设计思路 . . . . .      | 2  |
|   | 3.2 类结构设计 . . . . .     | 3  |
|   | 3.3 关键流程设计 . . . . .    | 4  |
| 4 | 关键实现说明 . . . . .        | 4  |
|   | 4.1 玩法封装 . . . . .      | 4  |
|   | 4.2 公共工具函数设计 . . . . .  | 5  |
|   | 4.3 输入合法性验证设计 . . . . . | 5  |
| 5 | 关键代码说明 . . . . .        | 6  |
|   | 5.1 抽象父类设计 . . . . .    | 6  |
|   | 5.2 主程序分发逻辑 . . . . .   | 6  |
|   | 5.3 和值玩法奖金映射 . . . . .  | 7  |
| 6 | 测试案例设计 . . . . .        | 7  |
|   | 6.1 测试思路 . . . . .      | 7  |
|   | 6.2 测试用例表 . . . . .     | 8  |
| 7 | 运行结果分析 . . . . .        | 9  |
| 8 | 心得与收获 . . . . .         | 9  |
| 9 | 结论 . . . . .            | 9  |
| 1 | 源代码 . . . . .           | 10 |

## 1 实验目的

本次实验以福彩 3D 模拟系统为背景，综合运用 Java 面向对象程序设计的相关知识，实现了不同投注方式下的奖金判定逻辑，并在此基础上完成了程序的测试与本实验报告的撰写。通过亲手完成这次实验，我进一步掌握了以下内容：

1. 抽象类、继承与多态在实际问题中的应用；
2. 不同玩法规则的统一建模方法；
3. 输入合法性验证与测试模块设计方法；

## 2 实验内容与需求分析

根据实验要求，我需要完成福彩 3D 模拟程序的设计与实现，主要涵盖了以下三部分内容：

1. 实现不同福彩玩法的中奖判断；
2. 针对不同玩法输入格式不同的问题，进行输入合法性校验；
3. 设计统一测试模板，便于对不同玩法进行测试。

实验说明中还要求测试模块能够让用户选择玩法、输入投注内容和倍数，并返回奖金。因此我意识到，本次实验的核心不只是单纯地“判断是否中奖”，而是需要构建一个具有统一接口、便于扩展和测试的面向对象程序框架。

## 3 总体设计

### 3.1 设计思路

我采用了“**抽象父类 + 多个具体子类**”的方式来实现不同的彩票玩法。设计思路如下：

1. 定义了抽象父类 Play，用于保存开奖号码并提供公共工具方法；
2. 针对每一种玩法分别定义了子类，如 Single、Group、OneD、TowD、General、PackagePlay、SumPlay 等；
3. 在每个子类中重写了 getMoney(String bet) 方法，专门处理该玩法自身的中奖逻辑；
4. 主程序会根据用户输入的玩法名称创建对应的对象，随后我通过调用统一接口 getMoney() 来计算奖金；

5. 通过父类中的公共方法复用了常见逻辑，例如：

- 判断三个数字是否全相同；
- 判断是否存在两个数字相同；
- 判断三个数字是否各不相同；
- 判断两个号码排序后是否相同；
- 计算和值；
- 判断是否为拖拉机号。

我选择这种设计方式主要看中了它的两个优点：一是不同玩法之间彼此独立，逻辑清晰；二是后续若增加新玩法，只需要新增相应的子类而不必修改已有代码，使程序具有较好的扩展性。

3.2 类结构设计

实验中设计的类结构如图 1 所示。使用父类 Play 抽象出了所有玩法共有的数据与行为，而各具体玩法子类则负责实现其自身的奖金计算规则。

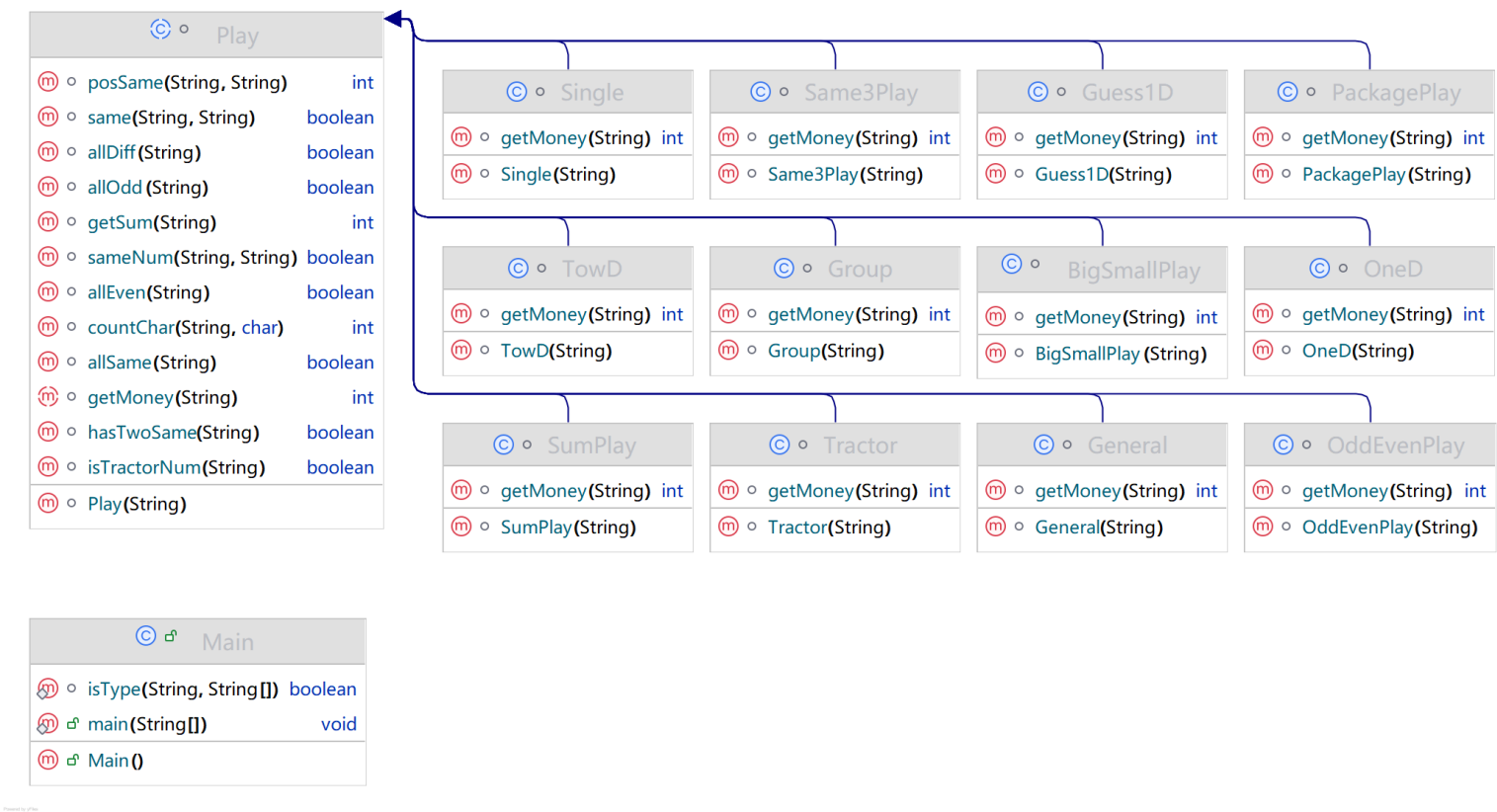
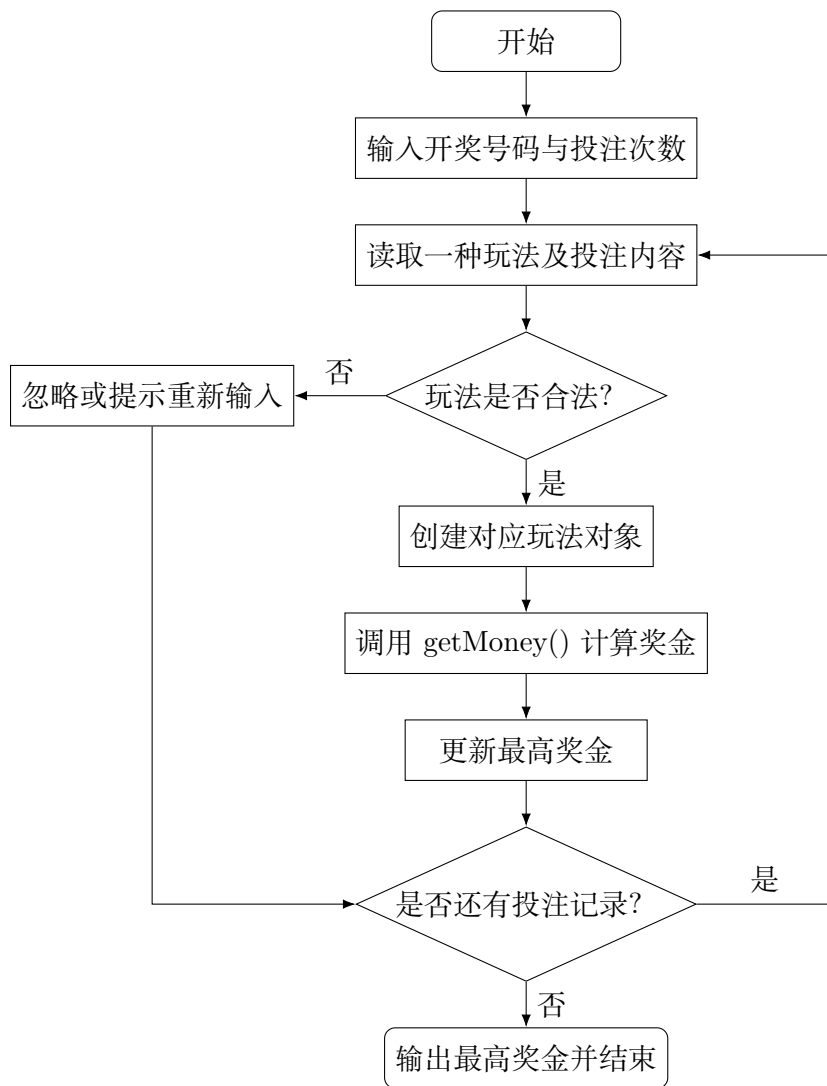


图 1 类结构

### 3.3 关键流程设计

根据代码梳理出的程序运行关键流程如图 3.3 所示。



## 4 关键实现说明

### 4.1 玩法封装

为了统一处理不同玩法，在父类中定义了抽象方法 `getMoney(String bet)`，并让各子类分别实现自己的中奖判断。我认为这样做能够很好地体现多态思想，使得主程序不必关心具体玩法的内部细节，只需要调用提供的统一方法即可。

例如：

- `Single` 类判断投注号码与开奖号码是否按位完全相同；

- Group 类根据是否为组选 3 或组选 6，分别判断中奖条件；
- OneD 类只检查指定位置上的数字是否相同；
- Guess1D 类统计所猜数字在开奖号码中出现的次数；
- TowD 类同时兼容 2D 投注与猜 2D 两种情况；
- PackagePlay 类根据号码特点自动区分包选三和包选六；
- SumPlay 类通过和值奖金表返回固定奖金。

## 4.2 公共工具函数设计

在编写过程中，我发现不同玩法之间有大量重复的判断逻辑，因此代码在父类中封装了多个公共工具函数。具体函数设计如表 1 所示：

表 1 父类公共工具函数设计说明

| 序号 | 函数签名            | 功能说明                 |
|----|-----------------|----------------------|
| 1  | same(a,b)       | 判断两个字符串是否完全相同        |
| 2  | sameNum(a,b)    | 将号码排序后比较，用于判断顺序无关的玩法 |
| 3  | allSame(s)      | 判断三个数字是否相同           |
| 4  | hasTwoSame(s)   | 判断是否恰有两个数字相同         |
| 5  | allDiff(s)      | 判断三个数字是否都不同          |
| 6  | posSame(a,b)    | 统计两个号码按位相同的个数        |
| 7  | getSum(s)       | 计算和值                 |
| 8  | isTractorNum(s) | 判断开奖号码是否为拖拉机号        |

将这些函数放入父类后，各子类就可以直接调用它们，这极大减少了代码冗余，也提高了程序的可维护性。

## 4.3 输入合法性验证设计

考虑到实验要求指出不同玩法具有不同的输入规则，我意识到需要进行严格的输入合法性校验。在设想的完整交互版程序中，我计划将输入校验设计为独立函数，例如：

- 单选、组选、包选、通选：必须输入长度为 3 的数字串；

- 1D: 必须输入长度为 3, 且其中两个字符为 \*;
- 猜 1D: 必须输入长度为 1 的数字;
- 2D 投注: 必须输入长度为 3, 且包含一个 \*;
- 猜 2D: 必须输入长度为 2 的数字;
- 和数: 必须输入 0~27 之间的整数;
- 猜大小: 只能输入 “大” 或 “小”;
- 猜奇偶: 只能输入 “奇” 或 “偶”。

## 5 关键代码说明

### 5.1 抽象父类设计

抽象父类是整个程序的核心, 它主要用于保存开奖号码并定义统一接口。它的基本作用就是 “描述所有玩法共有的属性和行为”。

Listing 1 抽象父类设计示意

```
1 abstract class Play {
2     String win;
3
4     Play(String win) {
5         this.win = win;
6     }
7
8     abstract int getMoney(String bet);
9 }
```

### 5.2 主程序分发逻辑

在主程序中, 根据玩法名称来创建相应对象, 再通过统一接口完成奖金计算。这样写避免了把所有玩法逻辑全部堆叠在 main 函数中, 使得整个程序的结构更加清晰。

Listing 2 主程序分发逻辑示意

```
1 if (type.equals("single")) {
2     p = new Single(win);
3     bet = sc.next();
```

```
4 } else if (type.equals("group")) {  
5     p = new Group(win);  
6     bet = sc.next();  
7 } else if (type.equals("oned")) {  
8     p = new OneD(win);  
9     bet = sc.next();  
10 } else if (type.equals("sum")) {  
11     p = new SumPlay(win);  
12     bet = sc.next();  
13 }
```

### 5.3 和值玩法奖金映射

对于和值玩法，其奖金与和值大小一一对应，因此选择使用条件判断表来实现。这部分代码直接根据和值返回固定的奖金，确保了逻辑清晰且便于后期校验。

Listing 3 和值奖金表示意

```
1 if (s == 0 || s == 27) return 1040;  
2 if (s == 1 || s == 26) return 345;  
3 if (s == 2 || s == 25) return 172;  
4 if (s == 3 || s == 24) return 104;  
5 ...  
6 return 14;
```

## 6 测试案例设计

### 6.1 测试思路

为了保证程序的正确性，我从以下几个方面设计了测试用例：

1. 正常中奖情况测试；
2. 不中奖情况测试；
3. 同一种玩法不同分支测试；
4. 临界值测试，例如和值最小值和最大值；
5. 特殊规则测试，例如拖拉机排除号码、豹子号、奇偶全同等。



## 6.2 测试用例表

表 2 中列出了部分具有代表性的测试用例。

表 2 测试用例设计表

| 编号 | 玩法      | 开奖号码 | 投注内容                   | 预期结果                       |
|----|---------|------|------------------------|----------------------------|
| 1  | single  | 745  | 745                    | 按位完全相同, 奖金为 1040           |
| 2  | group   | 121  | 112                    | 属于组选 3, 顺序不同但号码相同, 奖金为 346 |
| 3  | group   | 321  | 123                    | 属于组选 6, 顺序不同但号码相同, 奖金为 173 |
| 4  | oned    | 745  | *4*                    | 十位命中, 奖金为 10               |
| 5  | guess1d | 111  | 1                      | 猜中 3, 奖金为 230              |
| 6  | towd    | 512  | *12                    | 后两位按位命中, 奖金为 104           |
| 7  | towd    | 211  | 11                     | 猜 2D 两同号命中, 奖金为 37         |
| 8  | towd    | 521  | 12                     | 猜 2D 两不同号命中, 奖金为 19        |
| 9  | general | 123  | 12* (对应输入 123 与开奖号码比较) | 若恰有两位按位相同, 则奖金为 21         |
| 10 | sum     | 002  | 2                      | 和值为 2, 奖金为 172             |
| 11 | package | 110  | 011                    | 包选三组中, 奖金为 173             |
| 12 | package | 123  | 123                    | 包选六全中, 奖金为 606             |
| 13 | tractor | 456  | 无                      | 为升序连续数, 奖金为 65             |
| 14 | tractor | 890  | 无                      | 虽连续但属于排除号, 奖金为 0           |
| 15 | 猜大小     | 987  | 大                      | 和值为 24, 属于“大”, 奖金为 6       |
| 16 | 猜三同     | 777  | 无                      | 开出豹子号, 奖金为 104             |
| 17 | 猜奇偶     | 135  | 奇                      | 全为奇数, 奖金为 8                |

## 7 运行结果分析

从最终的测试结果可以看出，编写的程序能够正确区分不同玩法，并根据相应的玩法规则返回对应奖金。对于需要顺序匹配的玩法（如单选、1D、2D、通选等），我采用了按位比较的方式进行处理；对于不要求顺序的玩法（如组选、包选、猜 2D 两不同号等），则采用了排序比较或出现次数统计的方式来处理；对于和值、猜大小、猜三同、猜奇偶等特殊玩法，通过提取和值、奇偶性和重复性等特征完成了最终判断。

此外，我发现在采用面向对象设计后，程序的结构变得更加清晰了。主程序现在只负责输入与调度，不同玩法的核心规则均由各自子类独立完成，非常便于后续的测试和维护。

## 8 心得与收获

通过本次实验，我对 Java 中继承、多态和抽象类的使用有了更加直观的理解。以往在编写程序时，我往往习惯把所有逻辑集中在主函数中，虽然短时间内可以完成任务，但一旦功能增多，代码就会变得臃肿且难以维护。本次实验中，我将“不同玩法”抽象为“不同子类”，把共同逻辑提取到父类中，明显感受到了面向对象设计在程序组织方面的优势。

同时，我也认识到在实际开发中，“实现功能”并不是唯一目标。对于同一个问题，还需要考虑输入格式差异、错误输入处理、统一测试方式、代码复用性和后续扩展能力。这些内容虽然不直接影响某个测试点是否通过，但却直接影响了程序是否规范、是否便于维护。

在编写实验报告的过程中，我进一步体会到：写程序和写文档同样重要。通过梳理设计思路、绘制流程图、整理测试案例，我对自己程序的结构和实现过程有了更清晰的认识，也提高了总结与表达技术方案的能力。

## 9 结论

在本次实验中顺利完成了福彩 3D 模拟系统的基本设计与实现。程序能够根据不同玩法准确完成奖金判断，并且成功采用了抽象类与继承机制对各类玩法进行了统一管理。经过设计与调试，目前程序在结构上具备了较好的可读性和扩展性，这也为我后续加入输入合法性验证、倍数计算和完整交互测试模块打下了良好的基础。

## 1 源代码

Listing 4 福彩 3D 模拟程序源代码

```
1 import java.util.*;
2
3 abstract class Play {
4     String win;
5
6     Play(String win) {
7         this.win = win;
8     }
9
10    abstract int getMoney(String bet);
11
12    boolean same(String a, String b) {
13        return a.equals(b);
14    }
15
16    boolean sameNum(String a, String b) {
17        char[] x = a.toCharArray();
18        char[] y = b.toCharArray();
19        Arrays.sort(x);
20        Arrays.sort(y);
21        return new String(x).equals(new String(y));
22    }
23
24    boolean allSame(String s) {
25        return s.charAt(0) == s.charAt(1) && s.charAt(1) == s.charAt(2);
26    }
27
28    boolean hasTwoSame(String s) {
29        return s.charAt(0) == s.charAt(1) || s.charAt(0) == s.charAt(2)
30            || s.charAt(1) == s.charAt(2);
31    }
32
33    boolean allDiff(String s) {
34        return !hasTwoSame(s);
35    }
36 }
```

```
34     }
35
36     int posSame(String a, String b) {
37         int c = 0;
38         for (int i = 0; i < 3; i++) {
39             if (a.charAt(i) == b.charAt(i)) {
40                 c++;
41             }
42         }
43         return c;
44     }
45
46     int countChar(String s, char ch) {
47         int c = 0;
48         for (int i = 0; i < s.length(); i++) {
49             if (s.charAt(i) == ch) {
50                 c++;
51             }
52         }
53         return c;
54     }
55
56     int getSum(String s) {
57         return (s.charAt(0) - '0') + (s.charAt(1) - '0') + (s.charAt(2) -
58             '0');
59     }
60
61     boolean isTractorNum(String s) {
62         if (s.equals("890") || s.equals("098") || s.equals("901") || s.
63             equals("109")) {
64             return false;
65         }
66         int a = s.charAt(0) - '0';
67         int b = s.charAt(1) - '0';
68         int c = s.charAt(2) - '0';
69
70         if (b == a + 1 && c == b + 1) return true;
71         if (b == a - 1 && c == b - 1) return true;
```

```
70         return false;
71     }
72
73     boolean allOdd(String s) {
74         for (int i = 0; i < 3; i++) {
75             int x = s.charAt(i) - '0';
76             if (x % 2 == 0) return false;
77         }
78         return true;
79     }
80
81     boolean allEven(String s) {
82         for (int i = 0; i < 3; i++) {
83             int x = s.charAt(i) - '0';
84             if (x % 2 != 0) return false;
85         }
86         return true;
87     }
88 }
89
90 class Single extends Play {
91     Single(String win) {
92         super(win);
93     }
94
95     int getMoney(String bet) {
96         if (same(win, bet)) return 1040;
97         return 0;
98     }
99 }
100
101 class Group extends Play {
102     Group(String win) {
103         super(win);
104     }
105
106     int getMoney(String bet) {
107         if (bet.length() != 3) return 0;
```

```
108         if (allSame(bet)) return 0;
109
110         if (hasTwoSame(bet) && !allSame(bet)) {
111             if (hasTwoSame(win) && !allSame(win) && sameNum(win, bet)) {
112                 return 346;
113             }
114         } else if (allDiff(bet)) {
115             if (allDiff(win) && sameNum(win, bet)) {
116                 return 173;
117             }
118         }
119
120         return 0;
121     }
122 }
123
124 class OneD extends Play {
125     OneD(String win) {
126         super(win);
127     }
128
129     int getMoney(String bet) {
130         for (int i = 0; i < 3; i++) {
131             if (bet.charAt(i) != '*' && bet.charAt(i) == win.charAt(i)) {
132                 return 10;
133             }
134         }
135         return 0;
136     }
137 }
138
139 class Guess1D extends Play {
140     Guess1D(String win) {
141         super(win);
142     }
143
144     int getMoney(String bet) {
145         char ch = bet.charAt(0);
```

```
146         int c = countChar(win, ch);
147
148         if (c == 1) return 2;
149         if (c == 2) return 12;
150         if (c == 3) return 230;
151         return 0;
152     }
153 }
154
155 class TowD extends Play {
156     TowD(String win) {
157         super(win);
158     }
159
160     int getMoney(String bet) {
161         // 2D投注: 像 *12、1*2、12*
162         if (bet.length() == 3 && bet.indexOf('*') != -1) {
163             int cnt = 0;
164             for (int i = 0; i < 3; i++) {
165                 if (bet.charAt(i) != '*' && bet.charAt(i) == win.charAt(i)) {
166                     cnt++;
167                 }
168             }
169             if (cnt == 2) return 104;
170             return 0;
171         }
172
173         // 猜2D: 像 11 或 12
174         if (bet.length() == 2) {
175             char a = bet.charAt(0);
176             char b = bet.charAt(1);
177
178             // 两同号
179             if (a == b) {
180                 if (countChar(win, a) >= 2) return 37;
181                 return 0;
182             }
```

```
183
184         // 两不同号
185         if (countChar(win, a) >= 1 && countChar(win, b) >= 1) {
186             return 19;
187         }
188     }
189
190     return 0;
191 }
192 }
193
194 class General extends Play {
195     General(String win) {
196         super(win);
197     }
198
199     int getMoney(String bet) {
200         if (same(win, bet)) return 470;
201         if (posSame(win, bet) == 2) return 21;
202         return 0;
203     }
204 }
205
206 class SumPlay extends Play {
207     SumPlay(String win) {
208         super(win);
209     }
210
211     int getMoney(String bet) {
212         int x = Integer.parseInt(bet);
213         int s = getSum(win);
214
215         if (x != s) return 0;
216
217         if (s == 0 || s == 27) return 1040;
218         if (s == 1 || s == 26) return 345;
219         if (s == 2 || s == 25) return 172;
220         if (s == 3 || s == 24) return 104;
```



```
221         if (s == 4 || s == 23) return 69;
222         if (s == 5 || s == 22) return 49;
223         if (s == 6 || s == 21) return 37;
224         if (s == 7 || s == 20) return 29;
225         if (s == 8 || s == 19) return 23;
226         if (s == 9 || s == 18) return 19;
227         if (s == 10 || s == 17) return 16;
228         if (s == 11 || s == 16) return 15;
229         if (s == 12 || s == 15) return 15;
230         return 14; // 13 或 14
231     }
232 }
233
234 class PackagePlay extends Play {
235     PackagePlay(String win) {
236         super(win);
237     }
238
239     int getMoney(String bet) {
240         if (bet.length() != 3) return 0;
241         if (allSame(bet)) return 0;
242
243         // 包选3
244         if (hasTwoSame(bet) && !allSame(bet)) {
245             if (same(win, bet)) return 693;
246             if (hasTwoSame(win) && !allSame(win) && sameNum(win, bet))
247                 return 173;
248         }
249
250         // 包选6
251         if (allDiff(bet)) {
252             if (same(win, bet)) return 606;
253             if (allDiff(win) && sameNum(win, bet)) return 86;
254         }
255
256         return 0;
257     }
258 }
```

```
258
259 class BigSmallPlay extends Play {
260     BigSmallPlay(String win) {
261         super(win);
262     }
263
264     int getMoney(String bet) {
265         int s = getSum(win);
266
267         if ((bet.equals("大") || bet.equalsIgnoreCase("big")) && s >= 19
268             && s <= 27) {
269             return 6;
270         }
271         if ((bet.equals("小") || bet.equalsIgnoreCase("small")) && s >= 0
272             && s <= 8) {
273             return 6;
274         }
275         return 0;
276     }
277 }
278
279 class Same3Play extends Play {
280     Same3Play(String win) {
281         super(win);
282     }
283
284     int getMoney(String bet) {
285         if (allSame(win)) return 104;
286         return 0;
287     }
288 }
289
290 class Tractor extends Play {
291     Tractor(String win) {
292         super(win);
293     }
294
295     int getMoney(String bet) {
```

```
294         if (isTractorNum(win)) return 65;
295         return 0;
296     }
297 }
298
299 class OddEvenPlay extends Play {
300     OddEvenPlay(String win) {
301         super(win);
302     }
303
304     int getMoney(String bet) {
305         if ((bet.equals("奇") || bet.equalsIgnoreCase("odd")) && allOdd(
306             win)) {
307             return 8;
308         }
309         if ((bet.equals("偶") || bet.equalsIgnoreCase("even")) && allEven(
310             win)) {
311             return 8;
312         }
313         return 0;
314     }
315 }
316
317 public class Main {
318     static boolean isType(String s, String... arr) {
319         for (String x : arr) {
320             if (s.equals(x)) return true;
321         }
322         return false;
323     }
324
325     public static void main(String[] args) {
326         Scanner sc = new Scanner(System.in);
327
328         String win = sc.next();
329         int n = sc.nextInt();
330
331         int ans = 0;
```

```
330
331     for (int i = 0; i < n; i++) {
332         String type = sc.next();
333         Play p = null;
334         String bet = "";
335
336         if (isType(type, "single", "单选")) {
337             p = new Single(win);
338             bet = sc.next();
339         } else if (isType(type, "group", "组选")) {
340             p = new Group(win);
341             bet = sc.next();
342         } else if (isType(type, "oned", "1D")) {
343             p = new OneD(win);
344             bet = sc.next();
345         } else if (isType(type, "guess1d", "猜1D")) {
346             p = new Guess1D(win);
347             bet = sc.next();
348         } else if (isType(type, "towd", "2D", "猜2D")) {
349             p = new TowD(win);
350             bet = sc.next();
351         } else if (isType(type, "general", "通选")) {
352             p = new General(win);
353             bet = sc.next();
354         } else if (isType(type, "sum", "和数")) {
355             p = new SumPlay(win);
356             bet = sc.next();
357         } else if (isType(type, "package", "包选")) {
358             p = new PackagePlay(win);
359             bet = sc.next();
360         } else if (isType(type, "bigsmall", "guessbigsmall", "猜大小")) {
361             p = new BigSmallPlay(win);
362             bet = sc.next();
363         } else if (isType(type, "same3", "guesssame3", "猜三同")) {
364             p = new Same3Play(win);
365         } else if (isType(type, "tractor", "拖拉机")) {
366             p = new Tractor(win);
```

```
367         } else if (isType(type, "oddEven", "oddeven", "guessoddeven",  
368             "猜奇偶")) {  
369             p = new OddEvenPlay(win);  
370             bet = sc.next();  
371         }  
372         if (p != null) {  
373             ans = Math.max(ans, p.getMoney(bet));  
374         }  
375     }  
376     System.out.println(ans);  
377 }  
378 }  
379 }
```